



Push_swap

Because Swap_push doesn't feel as natural

Summary:

This activity will make you sort data on a stack, with a limited set of instructions, using the lowest possible number of actions. To succeed you'll have to manipulate various types of algorithms and choose the most appropriate solution (out of many) for optimized data sorting.

This is a group activity to be completed by exactly 2 learners.

Version: 1.1

Chapter II

Common Instructions

- Your project must be written in C.
- Your project must be written in accordance with the Norm. If you have bonus files/functions, they are included in the norm check, and you will receive a 0 if there is a norm error.
- Your functions should not quit unexpectedly (segmentation fault, bus error, double free, etc.) except for undefined behavior. If this occurs, your project will be considered non-functional and will receive a 0 during the evaluation.
- All heap-allocated memory must be properly freed when necessary. Memory leaks will not be tolerated.
- If the subject requires it, you must submit a `Makefile` that compiles your source files to the required output with the flags `-Wall`, `-Wextra`, and `-Werror`, using `cc`. Additionally, your `Makefile` must not perform unnecessary relinking.
- Your `Makefile` must contain at least the rules `$(NAME)`, `all`, `clean`, `fclean` and `re`.
- To submit bonuses for your project, you must include a `bonus` rule in your `Makefile`, which will add all the various headers, libraries, or functions that are not allowed in the main part of the project. Bonuses must be placed in `_bonus.{c/h}` files, unless the subject specifies otherwise. The evaluation of mandatory and bonus parts is conducted separately.
- If your project allows you to use your `libft`, you must copy its sources and its associated `Makefile` into a `libft` folder. Your project's `Makefile` must compile the library by using its `Makefile`, then compile the project.
- We encourage you to create test programs for your project, even though this work **does not need to be submitted and will not be graded**. It will give you an opportunity to easily test your work and your peers' work. You will find these tests especially useful during your defence. Indeed, during defence, you are free to use your tests and/or the tests of the peer you are evaluating.
- Submit your work to the assigned Git repository. Only the work in the Git repository will be graded. If Deepthought is assigned to grade your work, it will occur

after your peer-evaluations. If an error happens in any section of your work during Deepthought's grading, the evaluation will stop.

Chapter IV

Introduction

The **Push swap** project is a very simple and highly straightforward algorithm project: data must be sorted.

You have at your disposal a set of integer values, 2 stacks, and a set of operations to manipulate both stacks.

Your goal? Write a C program called `push_swap` which calculates and displays on the standard output the smallest program, made of *Push swap language* operations, that sorts the integers received as arguments.

Easy?

We'll see...

Chapter V

Objectives

The goal of this project is to make you discover [algorithmic complexity](#) in a very concrete way.

Sorting numbers is easy; sorting them *fast* with only two stacks and a handful of allowed moves is another story. Sorting a fully random list and sorting an almost sorted list are also two extremely different things.

You will quickly see how the choice of algorithm can make the difference between a quick win and an endless scroll of operations.

Chapter VI

Mandatory part

VI.1 Group project requirements

- This project must be completed by exactly 2 learners working together.
- Both learners must contribute meaningfully to the project and understand all implemented algorithms.
- The repository must clearly indicate both learners' contributions in the README.md file.
- During the defense, both learners must be present and able to explain any part of the code.
- The project submission should include both learners' logins in the repository.

VI.2 The rules

- You have 2 [stacks](#) named **a** and **b**.
- At the beginning:
 - The stack **a** contains a random amount of negative and/or positive numbers without any duplicate.
 - The stack **b** is empty.
- The goal is to sort in ascending order numbers into stack **a**. To do so you have the following operations at your disposal:
 - sa** (**swap a**): Swap the first two elements at the top of stack **a**.
Do nothing if there is only one or no elements.
 - sb** (**swap b**): Swap the first two elements at the top of stack **b**.
Do nothing if there is only one or no elements.
 - ss** : **sa** and **sb** at the same time.
 - pa** (**push a**): Take the first element at the top of **b** and put it at the top of **a**.
Do nothing if **b** is empty.

- pb** (**push b**): Take the first element at the top of **a** and put it at the top of **b**.
Do nothing if **a** is empty.
- ra** (**rotate a**): Shift up all elements of stack **a** by one.
The first element becomes the last one.
- rb** (**rotate b**): Shift up all elements of stack **b** by one.
The first element becomes the last one.
- rr** : **ra** and **rb** at the same time.
- rra** (**reverse rotate a**): Shift down all elements of stack **a** by one.
The last element becomes the first one.
- rrb** (**reverse rotate b**): Shift down all elements of stack **b** by one.
The last element becomes the first one.
- rrr** : **rra** and **rrb** at the same time.

VI.3 Algorithm requirements

To enforce a rigorous understanding of algorithmic complexity (*time* and *space*), you **must implement four distinct sorting strategies** and integrate them into your `push_swap` program. Your program must be able to select a strategy at runtime based on the input configuration.

VI.3.1 Complexity model and operation constraints

All strategies are implemented in C and must generate sequences of `Push_swap` operations to perform the sorting. This means:

- Your C algorithms analyze the input and generate the appropriate sequence of operations: **sa**, **sb**, **ss**, **pa**, **pb**, **ra**, **rb**, **rr**, **rra**, **rrb**, **rrr**.
- The output of your strategy is the sequence of these operations that sorts the stack.
- When you state a complexity class, it must reflect the cost **measured in number of Push_swap operations generated**, not the theoretical complexity of a classical array-based algorithm.

VI.3.2 Disorder metric (mandatory)

In this subject, **disorder** is a number between 0 and 1 that tells how far your initial stack **a** is from being sorted.

If the numbers are already in the right order, the disorder is 0. If they are in the worst possible order, the disorder is 1. Anything in between means your stack is partly sorted, but still messy.

To calculate it, you can think of looking at all the possible pairs of numbers in the stack. Each time a bigger number appears before a smaller one, that pair counts as a *mistake*. The more mistakes you have, the closer the disorder is to 1.

```
function compute_disorder(stack a):  
    mistakes = 0  
    total_pairs = 0  
    for i from 0 to size(a)-1:  
        for j from i+1 to size(a)-1:  
            total_pairs += 1  
            if a[i] > a[j]:  
                mistakes += 1  
    return mistakes / total_pairs
```

You must measure the disorder **before** doing any moves.

VI.3.3 Required strategies

1. **Simple algorithm ($O(n^2)$):**

Implement at least **one** baseline algorithm in the $O(n^2)$ class. Examples include:

- Insertion sort adaptation
- Selection sort adaptation
- Bubble sort adaptation
- Simple min/max extraction methods

2. **Medium algorithm ($O(n\sqrt{n})$):**

Implement at least **one** algorithm in the $O(n\sqrt{n})$ class. Examples include:

- Chunk-based sorting (divide into $\sqrt{n}$ chunks)
- Block-based partitioning methods
- Bucket sort adaptations with $\sqrt{n}$ buckets
- Range-based sorting strategies

3. **Complex algorithm ($O(n \log n)$):**

Implement at least **one** algorithm in the $O(n \log n)$ class. Examples include:

- Radix sort adaptation (LSD or MSD)
- Merge sort adaptation using two stacks
- Quick sort adaptation with stack partitioning
- Heap sort adaptation
- Binary indexed tree approaches

4. **Custom adaptive algorithm (learner's design):** Design an **adaptive** strategy that selects different internal methods depending on the measured **disorder**. You are **not** constrained to any specific named algorithm; the internal techniques are entirely up to you. However, your design must respect the following **complexity targets** per regime (in the Push_swap operation model):

Low disorder: if $\text{disorder} < 0.2$, your chosen method must run in $O(n^2)$ time.

Medium disorder: if $0.2 \leq \text{disorder} < 0.5$, your chosen method must run in $O(n\sqrt{n})$ time.

High disorder: if $\text{disorder} \geq 0.5$, your chosen method must run in $O(n \log n)$ time.

You must document in your repository (e.g., `README.md`) the rationale for your thresholds, the internal techniques used in each regime, and a brief complexity argument (upper bounds) for time and space within the Push_swap model.

VI.4 Example

To illustrate the effect of some of these operations, let's sort a random list of integers. In this example, we'll consider that both stacks grow from the right.

```
-----
Init a and b:
2
1
3
6
5
8
- -
a b
-----
Exec sa:
1
2
3
6
5
8
- -
a b
-----
Exec pb pb pb:
6 3
5 2
8 1
- -
a b
-----
Exec ra rb (equiv. to rr):
5 2
8 1
6 3
- -
a b
-----
Exec rra rrb (equiv. to rrr):
6 3
5 2
8 1
- -
a b
-----
Exec sa:
5 3
6 2
8 1
- -
a b
-----
Exec pa pa pa:
1
2
3
5
6
8
- -
a b
-----
```

The integers in stack a get sorted in 12 operations. Can you do better?

VI.5 The "push_swap" program

Program Name	push_swap
Files to Submit	Makefile, *.h, *.c
Makefile	NAME, all, clean, fclean, re
Arguments	stack a: A list of integers
External Function	• read, write, malloc, free, exit • ft_printf or any equivalent YOU coded
Libft authorized	Yes
Description	Sort stacks

Your project must comply with the following rules:

- You have to turn in a **Makefile** which will compile your source files. It must not relink.
- Global variables are forbidden.
- You must write a program named **push_swap** that takes as arguments:
 - The stack **a** formatted as a list of integers (the first argument is the top of the stack).
 - An optional **strategy selector**:
 - simple** Forces the use of your $O(n^2)$ algorithm.
 - medium** Forces the use of your $O(n\sqrt{n})$ algorithm.
 - complex** Forces the use of your $O(n \log n)$ algorithm.
 - adaptive** Forces the use of your adaptive algorithm based on **disorder**. This is the default behavior if no selector is given.
- The program must display the smallest list of **Push_swap** operations possible to sort stack **a**, the smallest number being at the top.
- Operations must be separated by a `\n` and nothing else.
- The complexity class claimed for each algorithm must be valid in this model.
- The strategy selection must work for **all valid inputs**. Any selector flag should work regardless of input size or disorder.
- If no parameters are specified, the program must not display anything and give the prompt back.

- In case of error, it must display **"Error"** followed by a `\n` on the standard error. Errors include, for example: arguments that are not integers, integers outside the valid range, or duplicate values.
- Your binary must embed all four strategies (Simple $O(n^2)$, Medium $O(n\sqrt{n})$, Complex $O(n \log n)$, and Adaptive). The selected strategy name and complexity class must be available in `--bench` mode.
- The optional **benchmark mode** (`--bench`) must display, after sorting:
 - The computed **disorder** (% with two decimals).
 - The name of the strategy used and its theoretical complexity class.
 - The total number of operations.
 - The count of each operation type (`sa`, `sb`, `ss`, `pa`, `pb`, `ra`, `rb`, `rr`, `rra`, `rrb`, `rrr`).

The benchmark output must be sent to `stderr` and only appear when the flag is present.

VI.5.1 Usage examples

Notation: lines prefixed with `[bench]` represent messages printed by the optional benchmark mode (to `stderr`). The operation stream remains on `stdout`.

```
$> ./push_swap 2 1 3 6 5 8
ra
pb
rra
pb
pb
ra
pb
ra
pb
pb
pa
pa
pa
pa
pa
```

Default selection (`--adaptive`) and operation count:

```
$> ARG="4 67 3 87 23"; ./push_swap --adaptive $ARG | wc -l
13
```

Force the simple ($O(n^2)$) strategy:

```
$> ./push_swap --simple 5 4 3 2 1
rra
pb
rra
pb
rra
pb
ra
pb
pb
pa
pa
pa
pa
pa
```

Force the complex ($O(n \log n)$) strategy and verify with the checker:

```
$> ARG="4 67 3 87 23"; ./push_swap --complex $ARG | ./checker_linux $ARG
OK
```

push_swap with a large input:

```
$> shuf -i 0-9999 -n 500 > args.txt ; ./push_swap $(cat args.txt) | wc -l
6784
$>
```

Run with benchmark enabled; hide operations and show only metrics:

```
$>shuf -i 0-9999 -n 500 > args.txt ; ./push_swap --bench $(cat
args.txt) 2> bench.txt | ./checker_linux $(cat args.txt)
OK
$> cat bench.txt
[bench] disorder: 49.93%
[bench] strategy: Adaptive /  $O(n\sqrt{n})$ 
[bench] total_ops: 7997
[bench] sa: 0 sb: 0 ss: 0 pa: 500 pb: 500
[bench] ra: 4840 rb: 1098 rr: 0 rra: 0 rrb: 1059 rrr: 0
```

Pipe operations to the checker while saving benchmark to a file:

```
$> ARG="4 67 3 87 23"; ./push_swap --bench --adaptive $ARG 2>
bench.txt | ./checker_linux $ARG
OK
$> cat bench.txt
[bench] disorder: 40.00%
[bench] strategy: Adaptive /  $O(n\sqrt{n})$ 
[bench] total_ops: 13
[bench] sa: 0 sb: 0 ss: 0 pa: 5 pb: 5
[bench] ra: 2 rb: 1 rr: 0 rra: 0 rrb: 0 rrr: 0
```

Error management examples:

```
$> ./push_swap --adaptive 0 one 2 3
Error
$> ./push_swap --simple 3 2 3
Error
```

- Write a program named **checker** that takes as an argument the stack **a** formatted as a list of integers. The first argument should be at the top of the stack (be careful about the order). If no argument is given, it stops and displays nothing.
- It will then wait and read operations from the standard input, with each instruction followed by '\n'. Once all the instructions have been read, the program has to execute them on the stack received as an argument.
- If after executing those instructions, the stack **a** is actually sorted and the stack **b** is empty, then the program must display "OK" followed by a '\n' on the standard output.
- In every other case, it must display "KO" followed by a '\n' on the standard output.
- In case of error, you must display "Error" followed by a '\n' on the **standard error**. Errors include for example: some arguments are not integers, some arguments are bigger than an integer, there are duplicates, an instruction doesn't exist and/or is incorrectly formatted.

```
$>./checker 3 2 1 0
rra
pb
sa
rra
pa
OK
$>./checker 3 2 1 0
sa
rra
pb
KO
$>./checker 3 2 one 0
Error
$>./checker "" 1
Error
$>
```



You DO NOT have to reproduce the exact same behavior as the provided binary. It is mandatory to manage errors but it is up to you to decide how you want to parse the arguments.